Capstone 5 — Domain B: Production B2B Order System

The culminating capstone: build a production-grade autonomous B2B order management system with multi-layer memory, model routing, full observability, cost optimization, containerized deployment, and a 100-case evaluation harness.

Prerequisites

REQUIRED MODULES

Complete these modules before starting Capstone 5-B. Each one teaches a production capability you will integrate here:

- **M03–M05 — Tool Use & Structured Output:** Foundations for every tool call and data extraction in the pipeline
- **M09–M10 — RAG Pipeline:** Hybrid search on product catalogs and customer contracts
- **M11 — Multi-Layer Memory:** Working, episodic, and procedural memory tiers
- **M12 — ReAct Agents:** The reasoning loop pattern used by every agent in this pipeline
- **M14 — Multi-Agent Systems:** Agent-to-agent handoffs and orchestration patterns (Capstone 4-B foundation)
- **M17 — HITL & Guardrails:** Human-in-the-loop routing, confidence thresholds, and circuit breakers
- **M18 — Evaluation:** Building test suites, automated scoring, and accuracy measurement
- **M19 — Tracing:** Distributed tracing with spans, trace collectors, and structured logging
- **M20 — Monitoring:** Dashboards, alerting, and real-time metrics for production systems
- **M21 — Deployment:** Containerization, queue-based processing, and API design
- **M22 — Cost Optimization:** Model routing, prompt caching, and per-request cost tracking

You should also have completed **Capstone 4 — Domain B** (multi-agent B2B order lifecycle pipeline), as this capstone extends that project with production capabilities.

Project Brief

BUSINESS CONTEXT

You've built the pieces across Capstones 1–4: a status bot (C1), a catalog/contract RAG agent (C2), a ReAct exception resolver (C3), and a 4-agent pipeline (C4). Now you integrate *everything* into a production system that handles the full B2B order lifecycle autonomously — intake through delivery — while learning from past exceptions, optimizing costs, and providing real-time visibility into every decision.

The system processes 200+ orders daily. Simple status queries resolve in under 5 seconds via Haiku. Exception investigations complete in under 60 seconds via Sonnet. Complex multi-exception resolutions escalate to Opus. Episodic memory ensures the system never makes the same mistake twice: if a carrier delay at Memphis was resolved with a 5% discount last time, the system proposes the same resolution automatically next time.

Targets: <5s for status queries, <60s for exception resolution, <$0.30/interaction average, >90% resolution accuracy vs. human ops managers, <10% escalation rate.

SIX PRODUCTION PILLARS

1. **Multi-Layer Memory:** Working memory (current order context), episodic memory (past exception resolutions, customer satisfaction outcomes), procedural memory (carrier-specific handling rules learned over time).
2. **Advanced RAG:** Hybrid search on product catalogs + contracts with customer-specific re-ranking and contextual compression.
3. **Full Observability:** Distributed tracing on every order event. 8-panel operations dashboard.
4. **Cost Optimization:** Model routing by task complexity. Prompt caching for catalog lookups. WebSocket streaming for real-time status.
5. **Production Deployment:** Docker + Express.js + BullMQ + ChromaDB. WebSocket for real-time order status. Event-driven architecture.
6. **Evaluation Harness:** 100-case test suite: 30 standard flows, 20 status queries, 25 exceptions, 10 edge cases, 10 adversarial, 5 regression.

Environment Setup

WHAT YOU'LL BUILD

A production-grade autonomous B2B order management system with multi-layer memory, model routing, full observability, cost optimization, WebSocket streaming, and a 100-case evaluation harness. Time estimate: **4–6 hours** (difficulty: ★★★★★).

System Requirements

- **Python 3.10+** (check with `python --version`)
- **Node.js 18+** (check with `node --version`) — for TypeScript path
- **pip** (check with `pip --version`)
- **Docker** (optional, for deployment steps — check with `docker --version`)
- An **Anthropic API key** with access to Haiku, Sonnet, and Opus models

Install Dependencies

Run this single command block to create your project directory and install everything:

MACOS/LINUX · WINDOWS

MACOS/LINUX

```
mkdir production-b2b-orders && cd production-b2b-orders
python -m venv venv
source venv/bin/activate
pip install "anthropic>=0.30.0" chromadb pydantic fastapi uvicorn httpx pytest websockets
```

WINDOWS

```
mkdir production-b2b-orders && cd production-b2b-orders
python -m venv venv
venv\Scripts\activate
pip install "anthropic>=0.30.0" chromadb pydantic fastapi uvicorn httpx pytest websockets
```

Set Your API Key

MACOS/LINUX · WINDOWS

MACOS/LINUX

```
export ANTHROPIC_API_KEY=your-api-key-here

# Verify it's set:
echo $ANTHROPIC_API_KEY
```

WINDOWS

```
# Command Prompt:
set ANTHROPIC_API_KEY=your-api-key-here

# PowerShell:
$env:ANTHROPIC_API_KEY = "your-api-key-here"

# To make it permanent, use System Environment Variables in Settings
```

Checkpoint

Verify your setup by running: `python -c "import anthropic; print(anthropic.__version__)"`. You should see a version number like `0.39.0` or higher. If you get a `ModuleNotFoundError`, ensure your virtual environment is activated.

File Structure

Here is every file you will create in this capstone. Each file has a specific role in the production order management system:

DIRECTORY TREE

```
production-b2b-orders/
├── agents/
│   ├── intake_agent.py      # Parses POs (EDI 850, email, portal), validates against catalog
│   ├── fulfillment_agent.py # Checks inventory, plans shipments (single vs. split)
│   ├── exception_agent.py   # Detects anomalies — delays, pricing mismatches, quality holds
│   └── comms_agent.py       # Generates customer notifications with SLA compliance
├── memory/
│   ├── working.py           # Current order context — 6K token budget
│   ├── episodic.py          # Past exception resolutions with CSAT outcomes (ChromaDB)
│   └── procedural.py        # Learned carrier/customer handling rules
├── rag/
│   ├── catalog_search.py    # Product catalog + contract hybrid search
│   └── vector_store.py      # TF-IDF mock vector store (swap to ChromaDB in production)
├── observability/
│   ├── tracing.py           # Trace spans for every LLM call, tool call, and agent handoff
│   └── dashboard.py         # 8-panel operations dashboard — orders/hr, cost, SLA
├── routing/
│   └── model_router.py      # Task-to-model mapping: Haiku / Sonnet / Opus + cost tracking
├── guardrails/
│   ├── input_validator.py   # Schema validation, duplicate PO detection, sanitization
│   └── circuit_breaker.py   # Carrier API failure detection — switch to fallback polling
├── evaluation/
│   ├── runner.py            # 100-case test suite runner with per-category scoring
│   └── test_suite.json      # Test cases: 30 standard, 20 status, 25 exceptions, 10 edge, 10 adversarial, 5 regression
├── server/
│   └── websocket.py         # WebSocket endpoint for real-time PO status streaming
├── pipeline.py              # Main orchestrator — coordinates all agents
├── mock_data.py             # 10 realistic POs with line items, carriers, contracts
├── config.py                # Configuration: routing, memory, deployment, thresholds
├── Dockerfile               # Production container image
├── docker-compose.yml       # Full stack: API + Redis + ChromaDB
└── requirements.txt         # All Python dependencies
```

Domain Glossary

Multi-Layer Memory

Three memory tiers: working (current order context), episodic (past exception outcomes in vector DB), procedural (learned carrier/customer handling rules).

Model Routing

Directing tasks to optimal Claude model: Haiku for status queries (~$0.003), Sonnet for exception investigation (~$0.03), Opus for multi-exception analysis (~$0.10). Two baselines: ~65% cheaper than all-Opus and ~17% cheaper than all-Sonnet.

WebSocket Streaming

Persistent connection allowing real-time order status updates pushed to clients as events occur, instead of polling. Critical for B2B portals where buyers monitor shipments live.

BullMQ

A Redis-based job queue for Node.js. Handles async order processing, retry logic, priority ordering, and back-pressure when volume spikes. The production alternative to synchronous processing.

Event-Driven Architecture

Order state changes (confirmed → shipped → delivered) emit events. Agents subscribe to relevant events instead of polling. Decouples producers from consumers.

Back-Pressure

When incoming request rate exceeds processing capacity, the system signals the sender to slow down (HTTP 429). Prevents queue overflow and maintains response quality for active requests.

Evaluation Harness

Automated test suite that scores system decisions against known-good resolutions. The production AI equivalent of unit tests. Catches regressions before they reach customers.

Graceful Degradation

When a component fails (ChromaDB down, carrier API unreachable), the system continues with reduced capability. RAG falls back to keyword search; carrier data shows "checking" instead of crashing.

Architecture

SIX PRODUCTION PILLARS

MODEL ROUTING — TASK → OPTIMAL MODEL

OPERATIONS DASHBOARD — REAL-TIME METRICS

Mock Data Specification

SYSTEM CONFIGURATION

SYSTEM CONFIGURATION


```
{
  "model_routing": {
    "order_validation": "claude-haiku-4-5-20251001",
    "status_query": "claude-haiku-4-5-20251001",
    "exception_investigation": "claude-sonnet-4-6",
    "exception_analysis": "claude-sonnet-4-6",
    "complex_resolution": "claude-opus-4-7",
    "customer_communication": "claude-haiku-4-5-20251001"
  },
  "memory": {
    "working": { "max_context_tokens": 6000 },
    "episodic": {
      "collection": "order_episodes",
      "retention_days": 180,
      "example": {
        "episode_id": "OE-2024-04521",
        "exception_type": "carrier_delay",
        "root_cause": "Memphis hub weather delay",
        "resolution": "Proactive notification + 5% discount on next order",
        "customer_satisfaction": "positive",
        "lesson": "Memphis hub delays in March are common — proactively notify within 2 hours"
      }
    },
    "procedural": {
      "rules": [{
        "rule_id": "OR-001",
        "confidence": 0.90,
        "rule": "For carrier delays at Memphis hub during March, proactively check weather alerts and notify affected
customers within 2 hours",
        "created_from_episodes": ["OE-2024-04521", "OE-2024-04533", "OE-2024-04601"]
      }]
    }
  },
  "observability": {
    "latency_status_target_ms": 5000,
    "latency_exception_target_ms": 60000,
    "cost_per_interaction_target": 0.30,
    "escalation_rate_target": 0.10
  },
  "deployment": {
    "runtime": "Docker",
    "api": "Express.js",
    "queue": "BullMQ + Redis",
    "vector_db": "ChromaDB",
    "streaming": "WebSocket",
    "max_concurrent": 50
  },
  "evaluation_suite": {
    "total_cases": 100,
    "breakdown": {
      "standard_flow": 30,
      "status_query": 20,
      "exception_carrier_delay": 10,
      "exception_pricing": 10,
      "exception_quality": 5,
      "edge": 10,
      "adversarial": 10,
      "regression": 5
    }
  }
}
```

Step-by-Step Build Guide

You will build this system in 10 steps. Each step produces a runnable component you can test independently before integrating into the full pipeline. Steps 1–4 build core infrastructure. Steps 5–8 add production capabilities. Steps 9–10 wire everything together and validate.

Step 1: Create the Project Skeleton

What & Why: Create the directory structure and empty `__init__.py` files so Python can import across packages. Without this, you will get `ModuleNotFoundError` on every import.

MACOS/LINUX · WINDOWS

MACOS/LINUX

```
cd production-b2b-orders
mkdir -p agents memory rag observability routing guardrails evaluation server
touch agents/__init__.py memory/__init__.py rag/__init__.py \
      observability/__init__.py routing/__init__.py guardrails/__init__.py \
      evaluation/__init__.py server/__init__.py
```

WINDOWS

```
cd production-b2b-orders
mkdir agents memory rag observability routing guardrails evaluation server
type nul > agents\__init__.py
type nul > memory\__init__.py
type nul > rag\__init__.py
type nul > observability\__init__.py
type nul > routing\__init__.py
type nul > guardrails\__init__.py
type nul > evaluation\__init__.py
type nul > server\__init__.py
```

Run Command

```
python -c "import agents; import memory; import rag; print('All packages importable')"
```

Checkpoint

You should see `All packages importable`. If you get `ModuleNotFoundError`, check that every directory has an `__init__.py` file and you are running from the `production-b2b-orders/` root.

Step 2: Build the Mock Data Layer

What & Why: Create `mock_data.py` with 10 realistic POs containing line items, carrier tracking, contracts, and pre-seeded exception scenarios. Every subsequent step depends on this test data.

PYTHON

```python
# mock_data.py — B2B Order Mock Data
# WHAT,
#       3 customer contracts with tiered pricing, and carrier tracking events.
# WHY:  Every subsequent step (memory, routing, RAG, orchestration) imports
#       from this file. Having consistent, rich data avoids "it works on my
#       machine" issues and lets checkpoints produce deterministic output.

ORDERS = {
    "PO-2024-11234": {
        "customer": "Acme Industrial",
        "customer_id": "CUST-001",
        "status": "shipped",
        "order_date": "2024-09-15",
        "ship_date": "2024-09-18",

    # … (full source in the desktop HTML) …

}
```

```
IF __name__ == "__main__":
    print(f"{len(ORDERS)} orders, {len(CONTRACTS)} contracts, {len(CARRIERS)} carrier records loaded")
    FOR po_id, order IN ORDERS.items():
        print(f"  {po_id}: {order['customer']:<20s} status={order['status']:<15s} total=${order['total']:,.2f}")
```

Run Command

```
python mock_data.py
```

Checkpoint

You should see `10 orders, 3 contracts, 2 carrier records loaded` followed by a summary of each PO. If you get an `ImportError`, make sure the file is saved as `mock_data.py` in the project root (`production-b2b-orders/`).

You can also verify imports work from other modules:

Run Command

```
python -c "from mock_data import ORDERS, CONTRACTS; print(f'{len(ORDERS)} orders, {len(CONTRACTS)}
contracts loaded')"
```

Checkpoint

You should see `10 orders, 3 contracts loaded` . If the count is wrong, verify all entries in `mock_data.py` .

Step 3: Implement Multi-Layer Memory

⚠ **Forward Dependency**

Note: This step requires `MockVectorStore` . If you haven't built it yet, copy the `MockVectorStore` class from Step 5 first, save it as `rag/vector_store.py` , then return here.

What & Why: Create three files in `memory/` : `working.py` (manages current order within 6K token budget), `episodic.py` (stores past exception resolutions with CSAT outcomes in the vector store), and `procedural.py` (extracts rules from repeated episode patterns like "Memphis hub delays in March"). This is the core differentiator from Capstone 4-B — it lets the system learn from experience. You will need the `MockVectorStore` from the Solution Architecture section below.

Run Command

```
python -c "
from memory.episodic import OrderEpisodicMemory
mem = OrderEpisodicMemory()
results = mem.recall('Memphis hub carrier delay')
print(f'Recalled {len(results)} episodes')
for r in results: print(f'  {r[\"episode_id\"]}: {r[\"resolution\"]} (similarity:
{r[\"similarity\"]})')
"
```

Checkpoint

You should see at least 1 recalled episode for the Memphis hub delay with a resolution like `notify + 5% discount` . If you see 0 results, check that `_seed()` is called in the constructor and the similarity threshold is not too high (lower to 0.1 if needed).

Step 4: Build the Model Router

What & Why: Create `routing/model_router.py` that maps task types to optimal Claude models. Status queries go to Haiku (~$0.003), exception investigations to Sonnet (~$0.03), and complex multi-exception resolutions to Opus (~$0.10). Average per-interaction cost drops to ~$0.019 — that is ~65% cheaper than all-Opus ($0.10) and ~17% cheaper than all-Sonnet ($0.030). The Production Cost Model section reconciles both baselines.

Run Command

```
python -c "
from routing.model_router import route_to_model
for task in ['status_query', 'exception_investigation', 'complex_resolution']:
    r = route_to_model(task)
    print(f'{task:<30} → {r[\"model\"]:<30} est: \${r[\"estimated_cost\"]:.4f}')
"
```

Checkpoint

You should see status_query routed to Haiku, exception_investigation to Sonnet, and complex_resolution to Opus with increasing estimated costs. If all routes point to the same model, check your `TASK_ROUTING` dictionary.

Step 5: Add Advanced RAG for Catalogs & Contracts

What & Why: Create `rag/vector_store.py` (TF-IDF mock) and `rag/catalog_search.py` (hybrid search with customer-specific re-ranking). This upgrades from Capstone 2-B by ensuring contract results always rank above general catalog results for a given customer. Add prompt caching for the system prompt and tool definitions.

Run Command

```
python -c "
from rag.catalog_search import search_catalog
results = search_catalog('industrial bearings', customer_id='CUST-001')  # Acme Industrial
for r in results: print(f'{r[\"source\"]}: {r[\"text\"][:60]}... (score: {r[\"score\"]})')
"
```

Checkpoint

Contract-specific results for Acme Industrial (`CUST-001`) should rank above general catalog entries. If catalog results rank first, check your re-ranking logic in `catalog_search.py` .

Step 6: Add Observability Tracing

What & Why: Create `observability/tracing.py` with the `trace_span` decorator and `observability/dashboard.py` for the 8-panel metrics dashboard. Trace every LLM call, tool call, and agent handoff so you can debug slow resolutions and diagnose errors. Without tracing, debugging a 45-second resolution across 6 components is guesswork. See the Observability / Tracing section below for the complete implementation.

Run Command

```
python -c "
from observability.tracing import trace_span, trace_log
@trace_span('test.memory')
def test_recall():
    return [{'id': 'OE-001'}]
test_recall()
print(f'Traces collected: {len(trace_log)}')
print(f'Last span: {trace_log[-1][\"span\"]} | {trace_log[-1][\"status\"]} | {trace_log[-1]
```

```
[\"duration_ms\"]}ms')
"
```

Checkpoint

You should see `Traces collected: 1` and a success span with a duration in milliseconds. If the trace log is empty, ensure the `trace_span` decorator appends to `trace_log`.

Step 7: Build the Four Agents

What & Why: Create the four specialized agents in `agents/`: (1) **Intake Agent** — parses POs, validates schema, detects duplicates; (2) **Fulfillment Agent** — checks inventory, plans single vs. split shipments, calculates ETAs; (3) **Exception Agent** — detects anomalies, queries episodic memory for similar past exceptions, proposes resolutions; (4) **Comms Agent** — generates customer notifications with SLA compliance and tone matching per customer tier. Each agent uses the model router from Step 4 and the tracing decorator from Step 6.

This step uses the memory from Step 3, router from Step 4, RAG from Step 5, and tracing from Step 6. If you skipped any step, complete it first.

Run Command

```
python -c "
from agents.intake_agent import IntakeAgent
from mock_data import ORDERS
agent = IntakeAgent()
result = agent.process(ORDERS[\"PO-2024-11234\"])
print(f'PO: {result[\"po_number\"]} | Status: {result[\"validation_status\"]} | Line items:
{result[\"line_count\"]}')
"
```

Checkpoint

You should see a validated PO with its line item count. If you get import errors, check that Steps 3–6 are complete and all `__init__.py` files exist.

Step 8: Wire Up the Pipeline Orchestrator

What & Why: Create `pipeline.py` that coordinates all four agents in sequence: intake → fulfillment → exception monitoring → customer communication. The orchestrator passes working memory context between agents, manages the circuit breaker (if carrier API fails >5 times in 10 minutes, switch to fallback polling), and applies the HITL gate for split shipments over $50K.

Run Command

```
python -c "
from pipeline import OrderPipeline
from mock_data import ORDERS
pipeline = OrderPipeline()
result = pipeline.process_order(ORDERS[\"PO-2024-11234\"])
print(f'Order {result[\"po_number\"]}: {result[\"final_status\"]}')
print(f'Agents invoked: {result[\"agents_invoked\"]}')
print(f'Total cost: \${result[\"total_cost\"]:.4f}')
"
```

Checkpoint

You should see the order processed through all four agents with a final status and total cost. The cost should be under $0.30 for a standard order. If an agent fails, check the trace log for the failing span.

Step 9: Add WebSocket Streaming & Deployment Config

What & Why: Create `server/websocket.py` for real-time order status updates and the `Dockerfile` + `docker-compose.yml` for containerized deployment. B2B buyers expect real-time visibility in their portal — WebSocket replaces polling (2,400 requests/minute for 200 orders) with ~50 targeted pushes per day. See the WebSocket Handler and Deployment Configuration sections below for complete implementations.

Run Command (WebSocket test)

```
python -c "
from server.websocket import active_connections, broadcast_order_update
print(f'WebSocket module loaded. Connection map type: {type(active_connections).__name__}')
print('Ready to accept connections on /ws/orders/{{po_number}}')
"
```

Checkpoint

You should see the module load successfully. For a full WebSocket test, start the server with `uvicorn server.websocket:app --port 3000` and connect a WebSocket client to `ws://localhost:3000/ws/orders/PO-2024-11234`.

Step 10: Build the Evaluation Harness

What & Why: Create `evaluation/runner.py` and `evaluation/test_suite.json` with 100 test cases: 30 standard order flows, 20 status queries, 25 exceptions (10 carrier delay, 10 pricing, 5 quality/partial), 10 edge cases, 10 adversarial inputs, and 5 regression tests. This is your production quality gate — run it after every code change to catch regressions before they reach customers. See the Evaluation Runner section below for the complete implementation.

Run Command

```
python -c "
from evaluation.runner import run_evaluation, print_report
from evaluation.runner import build_test_suite, mock_pipeline
suite = build_test_suite()
results = run_evaluation(suite, mock_pipeline)
print_report(results)
"
```

Checkpoint

You should see an evaluation report with pass/fail/partial counts and per-category accuracy bars. Target: >90% on standard flows, >80% on exceptions. If accuracy is low, check your mock pipeline against the expected outputs in `test_suite.json`.

Solution Architecture

This capstone is too large for a single file walkthrough. We provide the key architectural components that differentiate it from Capstone 4-B.

MockVectorStore (TF-IDF Similarity Search)

PYTHON

PYTHON

```
IMPORT math, re
FROM collections import Counter

CLASS MockVectorStore:
    FUNCTION __init__(self):
        self.documents = []
        self.vectors = []
        self.idf = {}

    FUNCTION _tokenize(self, text):
        RETURN re.findall(r'\b\w+\b', text.lower())

    FUNCTION _compute_tf(self, tokens):
        count = Counter(tokens)

    # … (full source in the desktop HTML) …

                "metadata": doc["metadata"],
                "similarity_score": round(dot / (mq * md), 4)
            })
        results.sort(key=lambda x,
                     reverse=True)
        RETURN results[:top_k]
```

Hybrid Catalog Search with Customer Re-Ranking

`rag/catalog_search.py` implements the hybrid retrieval used by the Fulfillment and Exception agents. It combines a BM25-style keyword score with a cosine semantic score on bag-of-words vectors, then applies a customer-specific re-ranking boost so that contracts tied to `customer_id` always surface above generic catalog entries. The function signature matches the import used in Step 5: `search_catalog(query, customer_id, top_k=5)`.

PYTHON

PYTHON

```
FROM __future__ import annotations
IMPORT math, re
FROM collections import Counter

# ---- Mock corpus ----------------------------------------------------
# In production this is loaded from ChromaDB / Postgres.
_CATALOG = [
    {"chunk_id": "CAT-4892", "source": "catalog",
     "text": "SKU-4892 Industrial Valve Assembly 4 inch. List price $595. "
             "Lead time 14 days. Bulk discount available at 50+ units.",
     "customer_id": None},
    {"chunk_id": "CAT-7210", "source": "catalog",
     "text": "SKU-7210 Stainless Steel Bearing 2 inch. List price $62.50. "
             "Volume tier at 100+ units. In-stock at Detroit warehouse.",

    # … (full source in the desktop HTML) …

IF __name__ == "__main__":
    hits = search_catalog("industrial valve 4 inch",
                          customer_id="CUST-001", top_k=4)
    FOR h IN hits= "[CONTRACT-BOOST]" IF h["customer_match"] else "         "
        print(f"{flag} {h['source']:<8} {h['chunk_id']:<15} "
              f"score={h['score']:.3f}  {h['text'][:55]}...")
```

Expected Output (Python demo)

```
[CONTRACT-BOOST] contract CTR-001-V4892   score=0.812  Acme Industrial CTR-2024-001: SKU-4892
contract pri...
                 catalog  CAT-4892         score=0.541  SKU-4892 Industrial Valve Assembly 4 inch.
List p...
                 contract CTR-003-V4892   score=0.498  Consolidated Mfg CTR-2024-003: SKU-4892 valve
ass...
```

```
                  contract CTR-001-P3300   score=0.276  Acme Industrial CTR-2024-001: SKU-3300
contract p...
```

> 🎯 **What Just Happened?**
>
> Step 5 imports `from rag.catalog_search import search_catalog`. This module implements the function with hybrid retrieval (BM25 + cosine) and a +0.25 score boost when a candidate document's `customer_id` matches the query's `customer_id`. That guarantees Acme Industrial's contract terms outrank a generic catalog entry, which is the correct B2B behaviour: contract pricing always wins.

Episodic Memory for Exception Resolution

PYTHON

PYTHON

```
FROM rag.vector_store import MockVectorStore

CLASS OrderEpisodicMemory:
    FUNCTION __init__(self):
        self.store = MockVectorStore()
        self._seed()

    FUNCTION _seed(self):
        self.store.add("OE-2024-04521",
            "Carrier delay at Memphis hub. UPS weather delay March. "
            "Customer Acme Industrial. Resolution: proactive notification "
            "within 2 hours + 5% discount on next order. CSAT,
            {"exception_type": "carrier_delay", "carrier": "UPS",
             "hub": "Memphis", "resolution": "notify + 5% discount",

    # … (full source in the desktop HTML) …

                FOR r IN results IF r["similarity_score"] > 0.1]

    FUNCTION store_episode(self, episode_id, description,
                      metadata):
        self.store.add(episode_id, description, metadata)
        RETURN {"episode_id": episode_id, "stored": True}
```

Model Router

PYTHON

PYTHON

```
FROM enum import Enum

CLASS Model(Enum):
    HAIKU = "claude-haiku-4-5-20251001"
    SONNET = "claude-sonnet-4-6"
    OPUS = "claude-opus-4-7"

TASK_ROUTING = {
    "order_validation": Model.HAIKU,
    "status_query": Model.HAIKU,
    "customer_communication": Model.HAIKU,
    "exception_investigation": Model.SONNET,
    "exception_analysis": Model.SONNET,
    "contract_pricing_check": Model.SONNET,

    # … (full source in the desktop HTML) …

    ELIF complexity < 0.2 and base == Model.SONNET= Model.HAIKU
    costs = COST_PER_1K[base]
    est_cost = 2 * (costs["input"] + costs["output"])  # ~2K tokens
    RETURN {"model": base.value, "task": task_type,
            "estimated_cost": round(est_cost, 4),
            "latency_ms": {Model.HAIKU, Model.SONNET, Model.OPUS: 15000}[base]}
```

🎯 **What Just Happened?**

Two components differentiate this from Capstone 4-B: (1) **Episodic memory** that recalls past exception resolutions with customer satisfaction scores, so the system proposes proven resolutions; (2) **Model routing** that sends 58% of interactions to Haiku (status queries), keeping average cost at $0.019 vs. the $0.30 target.

With memory and routing in place, the remaining production pillars close the gap between a prototype and a production system: observability for debugging, evaluation for quality assurance, and WebSocket streaming for real-time client updates.

Observability / Tracing Wrapper

Every tool call and LLM invocation must be traced. This decorator captures function name, inputs, outputs, duration, and status — giving you a full audit trail for debugging slow resolutions or diagnosing errors.

PYTHON

PYTHON

```
IMPORT time
FROM functools import wraps

trace_log= []

FUNCTION trace_span(span_name):

    FUNCTION decorator(func):

        FUNCTION wrapper(*args, **kwargs):
            start = time.time()
            try= func(*args, **kwargs)
                duration = time.time() - start
                trace_log.append({

    # … (full source in the desktop HTML) …

response = call_llm("claude-haiku-4-5-20251001", "Check PO status")

# Inspect the trace log
FOR span IN trace_log:
    print(f"[{span['status'].upper()}] {span['span']}"
          f" | {span['function']} | {span['duration_ms']}ms")
```

Expected Output

```
[SUCCESS] memory.episodic | recallSimilarEpisodes | 0.12ms
[SUCCESS] llm.route | callLlm | 0.04ms
```

🎯 **What Just Happened?**

You wrapped every function with a trace span that records its name, inputs, outputs, duration, and success/failure status. In production, these traces feed the 8-panel operations dashboard — letting you spot slow carrier API calls (>2s), expensive Opus invocations, or failing memory lookups instantly. Without tracing, debugging a 45-second resolution across 6 components is guesswork.

Input Validator (Schema + Sanitization)

`guardrails/input_validator.py` sits in front of the Intake Agent. It validates the incoming PO schema, detects duplicate POs (replayed webhooks), and sanitizes free-text fields against SQL/prompt injection. The sanitizer never silently rewrites text that changes meaning — instead it flags suspicious payloads and routes them to the adversarial test path. This is the production guardrail that makes the 10 adversarial test cases pass.

PYTHON

PYTHON

```python
IMPORT re
FROM typing import Any

# Compile once at import
_SQL_PATTERNS = [
    re.compile(r"';\s*(DROP|DELETE|UPDATE|INSERT)\s+", re.I),
    re.compile(r"\bOR\s+'?1'?\s*=\s*'?1", re.I),
    re.compile(r"--\s*$", re.M),
    re.compile(r"/\*.*?\*/", re.S),
]
_INJECTION_PATTERNS = [
    re.compile(r"ignore\s+(all\s+)?previous\s+instructions", re.I),
    re.compile(r"you\s+are\s+now\s+(an?\s+)?(admin|root|developer)",
               re.I),

    # … (full source in the desktop HTML) …

    # If attacks present, quarantine notes from any LLM context
    IF attacks= "[QUARANTINED]"
        result["audit_log"] = {"po_number": po["po_number"],
                               "attacks": attacks,
                               "raw_payload_redacted": True}
    RETURN result
```

🎯 What Just Happened?

You wired the production input guardrail. Schema validation rejects malformed POs at the door. Duplicate detection makes the system idempotent under webhook retries. Attack detection flags SQL injection, prompt injection, XSS, path traversal, null bytes, and oversized payloads — quarantining the raw payload so it never enters an LLM context window. The 10 adversarial test cases all flow through this validator and emerge with `sanitized: true` and `instruction_followed: false`.

Circuit Breaker (Fallback Polling Mode)

`guardrails/circuit_breaker.py` protects the pipeline from cascading failures when the carrier API misbehaves. After 5 failures within a 10-minute rolling window the breaker trips: subsequent calls short-circuit to *fallback polling mode* — batched periodic queries instead of real-time per-event lookups — until a half-open probe succeeds.

PYTHON

```python
IMPORT time
FROM collections import deque

WINDOW_SECONDS = 600   # 10-min rolling window
THRESHOLD = 5          # failures per window
COOLDOWN_SECONDS = 60  # before half-open probe

CLASS CircuitBreaker:
    FUNCTION __init__(self, name= "carrier_api"):
        self.name = name
        self._failures= deque()
        self._state = "closed"          # closed | open | half_open
        self._opened_at= None


    # … (full source in the desktop HTML) …


FUNCTION _fallback_polling(tracking_id):

    RETURN {"tracking_id": tracking_id,
            "status": "from_polling_cache",
            "freshness_minutes": 5}
```

🎯 **What Just Happened?**

You wired the carrier-API safety valve. Five failures inside any 10-minute window flips the breaker open; calls then short-circuit to a polling cache fed by a 5-minute batch worker. After 60 seconds the breaker half-opens for a single probe; if that probe succeeds, it closes; if it fails, the cooldown restarts. This is the pattern that lets the order pipeline tolerate a 30-minute carrier outage with no cascading failures, just slightly stale shipment data.

Evaluation Runner

The 100-case test suite means nothing without a runner that executes each case, scores the output, and reports accuracy by category. This runner loads the suite, runs each case through your pipeline, and prints a summary showing where the system excels and where it falls short.

PYTHON

PYTHON

```
IMPORT json

FUNCTION evaluate_output(actual, expected):

    score = 0.0
    checks = 0
    FOR key IN expected:
        checks += 1
        IF key in actual and actual[key] == expected[key]:
            score += 1
    RETURN score / checks IF checks else 0.0

FUNCTION run_evaluation(test_suite, pipeline_fn):


    # … (full source in the desktop HTML) …


# Run all 100 cases
suite = build_test_suite()
print(f"Loaded {len(suite['cases'])} test cases")
results = run_evaluation(suite, mock_pipeline)
print_report(results)
```

Expected Output

```
==================================================
  EVALUATION REPORT
==================================================
  Pass: 2  |  Partial: 1  |  Fail: 0
  Accuracy: 66.7%  (target: 90%)

  status_query                 ████████▒▒▒▒▒▒▒▒ 50% (1/2)
  exception_carrier_delay      ████████████████ 100% (1/1)
==================================================
```

🎯 **What Just Happened?**

You built an automated evaluation runner that scores your system against the 100-case test suite. Each case is classified as pass (≥90% field match), partial (≥50%), or fail. The category breakdown shows exactly where the system is weak — if `exception_pricing` drops below 80%, you know to investigate your contract RAG pipeline or episodic memory for pricing resolutions. Run this after every code change to catch regressions before they reach production.

WebSocket Handler for Real-Time Order Streaming

B2B buyers expect real-time order status updates in their portal. This WebSocket endpoint lets clients subscribe to a specific PO number and receive push updates as the order moves through the pipeline — no polling required.

PYTHON (FASTAPI)

PYTHON (FASTAPI)

```
FROM fastapi import FastAPI, WebSocket, WebSocketDisconnect
IMPORT json

app = FastAPI()
active_connections, list[WebSocket]] = {}

FUNCTION order_status_stream(
    websocket, po_number):

    websocket.accept()

    # Track connection by PO number
    IF po_number not in active_connections= []
    active_connections[po_number].append(websocket)

    # … (full source in the desktop HTML) …

#   await broadcast_order_update("PO-2024-11234", {
#       "status": "shipped",
#       "carrier": "UPS",
#       "tracking": "1Z999AA10123456784",
#       "estimated_delivery": "2024-04-05"
#   })
```

🎯 **What Just Happened?**

You built a WebSocket endpoint that lets B2B portal clients subscribe to real-time updates for specific PO numbers. When the pipeline changes an order state (confirmed → shipped → delivered), it calls `broadcast_order_update()` to push the event to all connected clients instantly. This replaces polling (2,400 requests/minute for 200 orders) with ~50 targeted pushes per day. The connection cleanup on disconnect prevents memory leaks that would crash the server in production.

ERP Webhooks & Pub/Sub Event Bus

Production B2B systems are *event-driven*: when SAP or NetSuite changes an order's status, it fires a webhook to your service, which validates the signature, publishes the event to a topic, and lets independent subscribers (Exception Agent, Comms Agent, audit log) react in parallel. Below are three small modules — a webhook receiver, a publisher, and a subscriber — with a mock in-memory queue you can swap for Google Cloud Pub/Sub in production.

WHY THREE MODULES INSTEAD OF ONE

The webhook handler runs synchronously inside FastAPI (must return 200 within seconds or the ERP retries). The publisher hands the event off to a topic asynchronously. The subscriber processes events in a worker. Decoupling these three layers is the difference between losing events under load and processing 200 orders/day reliably.

PYTHON (FASTAPI)

PYTHON (FASTAPI)

```
FROM __future__ import annotations
IMPORT hmac, hashlib, json, os, logging
FROM fastapi import FastAPI, Request, HTTPException, Header
FROM server.pubsub_publisher import publish

log = logging.getLogger("webhook")
app = FastAPI()

WEBHOOK_SECRET = os.environ.get("ERP_WEBHOOK_SECRET", "")

FUNCTION _verify_signature(body, signature):
    IF not WEBHOOK_SECRET or not signature= hmac.new(
        WEBHOOK_SECRET.encode("utf-8"),
        body,

    # … (full source in the desktop HTML) …

        "po_number": event["po_number"],
        "status": event["status"],
        "occurred_at": event["occurred_at"],
        "raw": event,
    })
    RETURN {"accepted": True, "po_number": event["po_number"]}
```

PYTHON

PYTHON

```
FROM __future__ import annotations
FROM collections import defaultdict
FROM typing import Callable, Any
IMPORT logging, queue, threading

log = logging.getLogger("pubsub")

# Topic name, queue.Queue] = defaultdict(queue.Queue)
# Topic name, list[Callable[[dict], None]]] = defaultdict(list)
_lock = threading.Lock()

FUNCTION publish(topic, event, Any]):

    IF not topic:

    # … (full source in the desktop HTML) …


FUNCTION subscribe(topic, callback, None]):

    WITH _lock:
        _subscribers[topic].append(callback)
    log.info("subscriber registered for %s", topic)
```

PYTHON

```
FROM __future__ import annotations
IMPORT logging
FROM server.pubsub_publisher import subscribe

log = logging.getLogger("subscriber")

# Lazy imports avoid circular references during agent boot
FUNCTION _on_status_changed(event):

    status = event.get("status")
    po = event.get("po_number")
    log.info("dispatch po=%s status=%s", po, status)

    TRY:

    # … (full source in the desktop HTML) …

        "type": "erp.order.status_changed",
        "po_number": "PO-2024-12180",
        "status": "exception",
        "occurred_at": "2024-11-15T08:30,
        "raw": {"reason": "Memphis hub weather hold"},
    })
```

🎯 What Just Happened?

You wired the full event-driven path: ERP → signed webhook → `publish()` → in-process queue → `subscribe()` dispatch → the appropriate agent. Production-grade behaviour: HMAC verification, JSON validation, isolated error handling per subscriber, and an explicit production swap-in note for Google Cloud Pub/Sub. Run the smoke test with `python -m server.pubsub_subscriber` to see a fake exception event flow end-to-end.

Prompt Caching for the Product Catalog

The product catalog is large (~6K tokens) but rarely changes. Without caching, every status query and exception investigation re-reads it — that is wasted input cost. With `cache_control={"type": "ephemeral"}`, Anthropic stores the prefix server-side for 5 minutes and bills the next request that re-uses it at **~10% of normal input cost**. Caching activates when (a) the prefix is at least 1024 tokens for Haiku/Sonnet (or 2048 for Opus) and (b) the new request reuses that exact prefix. The dashboard's 82% cache-hit rate comes from this pattern.

PYTHON

```
FROM __future__ import annotations
IMPORT os
FROM anthropic import Anthropic, APIError

client = Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])

# In production this is loaded from the catalog table at boot.
CATALOG_TEXT = """\
You are a B2B order operations assistant for an industrial supplier.
Use ONLY the catalog and contract data below to answer.

PRODUCT CATALOG (authoritative)
- SKU-4892 Industrial Valve Assembly 4 inch | list $595 | 14d lead
- SKU-7210 Stainless Steel Bearing 2 inch  | list $62.50 | 7d lead

    # … (full source in the desktop HTML) …

IF __name__ == "__main__":
    # First call: cache_create_input_tokens > 0 (paying to write cache)
    answer_with_cached_catalog("PO-2024-11234", "Status of this order?")
    # Second call within 5 minutes: cache_read_input_tokens > 0
    answer_with_cached_catalog("PO-2024-11710",
                               "What is the lead time on SKU-3300?")
```

Expected Output (two consecutive calls)

```
input=42 cache_create=1280 cache_read=0     # first call writes cache
input=58 cache_create=0    cache_read=1280  # second call hits cache
```

> 💰 **WHAT THIS SAVES**
>
> Cached input tokens are billed at ~10% of normal input cost. With a 6K-token catalog and 6,000 monthly requests, caching saves roughly 6K × 6,000 × 0.9 = 32.4M token-equivalents of input. At Haiku's $0.0008/1K input price, that is ~$26/month off Haiku alone, plus larger savings on Sonnet calls. Combined with model routing, this is the path from $30/month all-Sonnet API cost down to ~$15/month.

Deployment Configuration

DOCKER-COMPOSE.YML · DOCKERFILE

DOCKER-COMPOSE.YML

```
version: "3.9"
services:
  api:
    build: .
    ports: ["3000:3000"]
    environment:
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
      - REDIS_URL=redis://redis:6379
      - CHROMA_URL=http://chromadb:8000
    depends_on: [redis, chromadb]
    deploy:
      resources:
        limits: { cpus: "2", memory: "2G" }

  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]

  chromadb:
    image: chromadb/chroma:latest
    ports: ["8000:8000"]
    volumes: ["chroma_data:/chroma/chroma"]

volumes:
  chroma_data:
```

DOCKERFILE

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 3000
CMD ["uvicorn", "server.websocket:app", "--host", "0.0.0.0", "--port", "3000", "--workers", "4"]
```

Evaluation Harness

TEST SUITE BREAKDOWN

TEST SUITE BREAKDOWN


```
{
  "total_cases": 100,
  "categories": {
    "standard_flow": {
      "count": 30,
      "description": "Normal orders: intake → fulfillment → ship → deliver",
      "example": "Single-warehouse order for Acme, all SKUs in stock"
    },
    "status_query": {
      "count": 20,
      "description": "Simple 'where's my order' queries",
      "example": "Check PO-2024-11234 status — should resolve via Haiku in <5s"
    },
    "exception_carrier_delay": {
      "count": 10,
      "description": "Carrier weather/logistics delays",
      "example": "Memphis hub delay — system should recall past resolution"
    },
    "exception_pricing": {
      "count": 10,
      "description": "Pricing discrepancies between order and contract",
      "example": "Invoiced at list $595 instead of contract $495"
    },
    "exception_quality": {
      "count": 5,
      "description": "Quality holds requiring replacement shipments",
      "example": "SKU-7721 on quality hold, replacement shipment dispatched"
    },
    "edge": {
      "count": 10,
      "description": "Missing/invalid line items, oversized, fractional, duplicate POs",
      "example": "Quantity=0, negative qty, expired contract, partial inventory"
    },
    "adversarial": {
      "count": 10,
      "description": "Injection attempts, malformed POs, extreme payloads",
      "example": "SQL/XSS/prompt injection — must sanitize and reject instructions"
    },
    "regression": {
      "count": 5,
      "description": "Pinned cases that previously broke — guard against regressions",
      "example": "PO-2024-11234, PO-2024-11301, etc."
    }
  },
  "scoring": {
    "exact_match": "Resolution matches expected action",
    "partial": "Correct diagnosis but suboptimal resolution",
    "miss": "Wrong diagnosis or harmful resolution",
    "target_accuracy": 0.90
  }
}
```

Testing Guide

TYPE	SCENARIO	EXPECTED BEHAVIOR
HAPPY	Standard order via Haiku	Processed end-to-end in <5s, costs <$0.05
HAPPY	Carrier delay + episodic memory	Recalls Memphis hub resolution, proposes same approach, resolves in <30s
HAPPY	Pricing discrepancy via Sonnet	Detects contract vs. invoice mismatch, calculates credit amount
HAPPY	20 concurrent status queries	All served from Haiku within 3s, queue handles gracefully
HAPPY	Procedural rule fires for Memphis delay	Proactive notification sent within 2 hours without human intervention
EDGE	Queue exceeds 100 orders	Priority ordering (high-value first), HTTP 429 for overflow
EDGE	Customer contract expires mid-processing	Detects stale cache, refreshes, uses list pricing with note
EDGE	Never-seen-before exception type	Routes to Opus, processes without episodic memory, stores as new episode
ADVERSARIAL	SQL injection in PO line item description	Sanitized at intake, no downstream impact
ADVERSARIAL	200 concurrent requests (4x max)	Back-pressure applied, 429 for overflow, no crashes

Compliance Notes

⚠ **PCI-DSS, EDI & COMMERCIAL CONFIDENTIALITY**

A production order management system handles sensitive commercial data across every component: customer credit limits, contract pricing, inventory levels, carrier rates, and exception resolution history.

- **Episodic memory as commercial data:** Past exception resolutions contain customer names, order values, discounts given, and satisfaction scores. Treat as confidential business records. Apply retention policies and access controls.
- **Cross-customer isolation:** The episodic memory must NEVER suggest a resolution based on Customer A's contract terms when resolving Customer B's exception. Filter episodes by customer before surfacing.
- **EDI compliance:** If orders arrive via EDI 850, every transformation must maintain the EDI audit trail. The evaluation harness should include EDI-formatted test cases.
- **Model routing transparency:** Log which model was selected for each interaction. If a customer dispute arises about a resolution quality, the ops team needs to know if it was handled by Haiku vs. Opus.
- **WebSocket security:** Real-time order status feeds must authenticate clients and only stream events for orders the authenticated user is authorized to see.

💰 **PRODUCTION COST MODEL**

With model routing: 58% Haiku (~$0.003), 35% Sonnet (~$0.03), 7% Opus (~$0.10). Average: ~$0.019/interaction (58% × $0.003 + 35% × $0.03 + 7% × $0.10 = $0.019). Prompt caching of the system prompt + catalog saves another ~30%: ~$0.015. Infrastructure (Redis, ChromaDB, compute): ~$300/month for 200 orders/day. Total: ~$400/month all-in for 6,000 monthly order interactions, or ~$0.07/interaction including infrastructure.

Two reference baselines:

- **All-Opus baseline (~$0.10/interaction):** 6,000 × $0.10 = $600/month API + $300 infra = $900/month. Routing saves ~$500/month or **~65% on API spend** ($0.019 vs $0.10).
- **All-Sonnet baseline (~$0.03/interaction):** 6,000 × $0.03 = $180/month API + $300 infra = $480/month. Routing saves ~$80/month or **~17% on API spend** ($0.019 vs $0.030) *and* raises quality for complex cases via Opus.

The 65% number is the marketing-friendly headline (most teams compare to "always use the smartest model"). The 17% number is the realistic operational improvement once you have already moved off Opus-for-everything. Both are correct against their respective baselines.

Verify Everything Works

Run this single end-to-end command that processes an order through the full pipeline, checks memory recall, verifies tracing, and runs the evaluation harness:

END-TO-END TEST

```
FROM pipeline import OrderPipeline
FROM mock_data import ORDERS
FROM memory.episodic import OrderEpisodicMemory
FROM observability.tracing import trace_log
FROM evaluation.runner import run_evaluation, build_test_suite, mock_pipeline, print_report

# 1. Process a standard order
pipeline = OrderPipeline()
result = pipeline.process_order(ORDERS["PO-2024-11234"])
print(f"[1/4] Order processed: {result['po_number']} → {result['final_status']}")
print(f"       Cost: ${result['total_cost']:.4f} | Agents: {result['agents_invoked']}")

# 2. Test episodic memory recall
mem = OrderEpisodicMemory()

    # … (full source in the desktop HTML) …

# 4. Run evaluation harness
print(f"\n[4/4] Running evaluation harness...")
eval_results = run_evaluation(build_test_suite(), mock_pipeline)
print_report(eval_results)

print("All systems operational.")
```

Run Command

```
python run_e2e_test.py
```

Expected Output

```
[1/4] Order processed: PO-2024-11234 → fulfilled
       Cost: $0.0096 | Agents: ['intake', 'fulfillment', 'comms']

[2/4] Episodic memory: 1 similar episodes recalled
       OE-2024-04521: notify + 5% discount

[3/4] Tracing: 6 spans captured
       [SUCCESS] agent.intake | 1.23ms
       [SUCCESS] agent.fulfillment | 0.89ms
       [SUCCESS] agent.comms | 0.54ms

[4/4] Running evaluation harness...
==================================================
  EVALUATION REPORT
==================================================
  Pass: 2  |  Partial: 1  |  Fail: 0
  Accuracy: 66.7%  (target: 90%)

  status_query                 ████████░░░░░░░░ 50% (1/2)
  exception_carrier_delay      ████████████████ 100% (1/1)
==================================================

All systems operational.
```

Congratulations!

You have built a production-grade autonomous B2B order management system with all six production pillars: multi-layer memory, advanced RAG, model routing, full observability, containerized deployment, and a 100-case evaluation harness. The pipeline processes orders through intake → fulfillment → exception monitoring → customer communication with tracing at every stage.

To take this further: connect real Anthropic API calls (replace mock responses in each agent), deploy with Docker using `docker-compose up`, and expand the evaluation harness to the full 100 cases to prove production readiness.

Troubleshooting Guide

COMMON ERRORS & FIXES

`ModuleNotFoundError: No module named 'anthropic'`
Your virtual environment is not activated. Run `source venv/bin/activate` (macOS/Linux) or `venv\Scripts\activate` (Windows), then `pip install anthropic`.

`ModuleNotFoundError: No module named 'agents'`
Python cannot find the package. Create `__init__.py` files in every subdirectory. On Unix: `touch agents/__init__.py memory/__init__.py rag/__init__.py observability/__init__.py routing/__init__.py guardrails/__init__.py evaluation/__init__.py server/__init__.py`. On Windows: `type nul > agents\__init__.py` (repeat for each directory).

`AuthenticationError: Invalid API key`
Your `ANTHROPIC_API_KEY` environment variable is not set or is incorrect. Run `echo $ANTHROPIC_API_KEY` (Unix) or `echo %ANTHROPIC_API_KEY%` (Windows) to verify. The key should start with `sk-ant-`.

Episodic memory returns 0 results for known exceptions
The mock TF-IDF embedding has lower resolution than real embeddings. Try lowering the similarity threshold from 0.1 to 0.05 in `memory/episodic.py`, or ensure your query includes key terms from the seeded episodes (e.g., "Memphis", "carrier", "delay").

Model router returns the same model for all tasks
Check that `TASK_ROUTING` in `routing/model_router.py` has distinct entries for each task type. Also verify the `complexity` parameter is not always above 0.85 (which forces Opus).

WebSocket connection refused on port 3000
Ensure the FastAPI server is running: `uvicorn server.websocket:app --port 3000`. If the port is already in use, try a different port: `uvicorn server.websocket:app --port 3001`. On Windows, check that your firewall allows inbound connections on the port.

Docker build fails with "pip install" errors
Create a `requirements.txt` file with: `anthropic chromadb pydantic fastapi uvicorn httpx pytest websockets` (one per line). Ensure it is in the project root directory.

Evaluation harness shows 0% accuracy on all categories
Check that your `mock_pipeline` function returns dictionaries with the same keys as the `expected` field in the test cases. Field name mismatches (e.g., `status` vs. `order_status`) cause 0-score evaluations.

Agent SDK Port [OPTIONAL STRETCH]

The four-agent pipeline you built uses a manual tool-use loop — `messages.create`, `stop_reason == "tool_use"`, accumulating `tool_result` blocks, and short-circuiting on `end_turn`. The **Claude Agent SDK** abstracts that loop. This stretch goal ports one agent (the Exception Resolution agent) so you can compare LOC and behavior against the manual implementation in your own code.

WHY PORT TO THE SDK?

The SDK trades fine-grained control for less code. Use it when your tools fit the MCP shape, async execution is acceptable, and circuit-breaker / retry logic can wrap the whole `query()` call rather than every iteration. Keep the manual loop when you need synchronous flow, mid-loop guardrails (e.g. cost ceilings checked between tool calls), or per-tool retry semantics.

Install

```
pip install "claude-agent-sdk≥0.1.0" anyio
```

Port the Exception Resolution Agent

Create `sdk_port.py`. The same exception-handling tools (`diagnose_exception` , `recall_similar_episodes` , `propose_resolution` , `escalate_to_human`) are wrapped with `@tool` and bundled into an in-process MCP server.

PYTHON

```
IMPORT anyio
IMPORT json
FROM claude_agent_sdk import (
    query,
    ClaudeAgentOptions,
    AssistantMessage,
    tool,
    create_sdk_mcp_server,
)
FROM agents.exception_tools import (
    diagnose_exception,
    recall_similar_episodes,
    propose_resolution,
    escalate_to_human,

    # … (full source in the desktop HTML) …

        state,
        ModelRouter(),
        Tracer(),
        CircuitBreaker(threshold=5, window_seconds=600),
    )
    print(out)
```

LOC & Behavior Comparison

ASPECT	MANUAL TOOL-USE LOOP	AGENT SDK (THIS STRETCH)
Lines per agent	~80	~25 (excluding tool wrappers)
Tool dispatch	You write the `if block.name == ...` dispatcher	SDK routes by MCP tool name
`stop_reason` / tool-use loop	You implement	SDK handles
Execution model	Sync (or your choice)	Async only
Circuit breaker hook	Inline, per iteration	Wrapper, per `query()` call
Tool result format	Plain dict you marshal	MCP `{"content": [...]}`
Mid-loop retry on tool failure	Easy — you control the loop	Harder — wrap the whole query
Visibility into raw messages	Full `messages` array	`AssistantMessage` event stream
Streaming partial responses	Set `stream=True` on `messages.create`	Native (the `async for`)
Per-step cost ceiling check	Easy — check after each tool result	Requires wrapping each tool with budget guard

DECISION HEURISTIC

Use the SDK when tools fit the MCP shape, the circuit breaker can guard the whole query (not every iteration), and async execution is acceptable. **Keep the manual loop when** you need synchronous control flow, per-tool-call retries, mid-loop budget enforcement, or you want complete visibility into the raw messages array (e.g. for replay, audit logs, or PCI-DSS trace inspection). The four-agent pipeline keeps the manual loop because the model router can switch tiers between iterations (Haiku → Sonnet → Opus on escalation) and the cost ceiling is checked after every tool call — both are easier to express in the explicit loop.